

Airport Route Optimization System

Spring Boot · Graph Algorithms · REST APIs

Data Structures

- Adjacency Matrix:
 - A 2D array where `matrix[i][j]` stores the distance from airport `i` to airport `j`.
 - If no direct flight exists, we store infinity.
 - For 24 airports, this creates a 24×24 matrix with 576 cells.
 - Space complexity: $O(V^2)$ - always uses V^2 cells regardless of how many routes exist.
- Adjacency List:
 - An array of lists where `adj[i]` contains only the actual neighbors of airport `i`.
 - For example, if JFK connects to 5 airports, `adj[JFK]` has only 5 entries, not 24.
 - Space complexity: $O(V + E)$ - only stores vertices plus actual edges

Phase 1 - Adjacency Matrix

- BFS
 - for v in range(n): # Checks all 24 airports
 - if $\text{graph}[u][v] < \text{INF}$ and not visited[v]:
- DFS
- Dijkstra
 - While priority queue not empty:
 - Extract minimum distance airport: $O(\log V)$
 - Scan ALL 24 matrix cells for neighbors: $O(V)$
 - Update distances and push to queue: $O(\log V)$

Phase 2 - Adjacency List

- BFS
 - for v, w in adj[u]: # Only checks actual neighbors!
 - if not visited[v]:
 - # Process neighbor
- DFS
 - def dfs(u):
 - visited[u] = True
 - for v, w in adj[u]: # Only actual neighbors
 - if not visited[v]:
 - dfs(v)

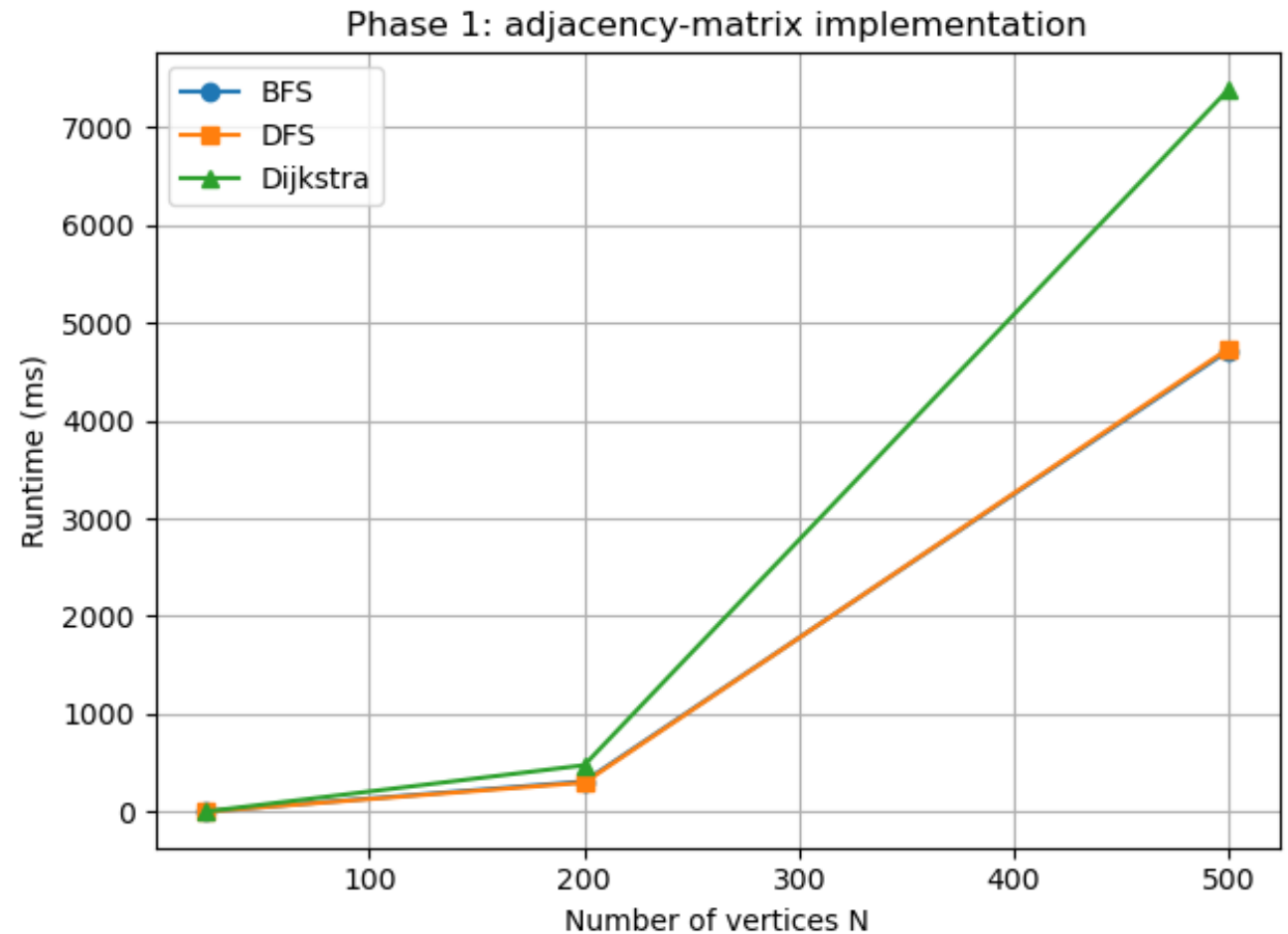
Algorithm	Phase 1 (Matrix)	Phase 2 (List)	Improvement
BFS	$O(V^2)$	$O(V + E)$	$V^2 \rightarrow (V+E)$
DFS	$O(V^2)$	$O(V + E)$	$V^2 \rightarrow (V+E)$
Dijkstra	$O(V^2 \log V)$	$O((V+E) \log V)$	$V^2 \rightarrow (V+E)$

When to use each approach

- Dense graphs (density > 50%)
 - Example: Complete graphs where everyone connects to everyone
 - Matrix and list use similar space at 50% density
- Frequent edge queries
 - Need to ask 'Does route $X \rightarrow Y$ exist?' many times
 - Matrix: $O(1)$ lookup
 - List: $O(\text{degree})$ scan
- Small graphs ($V < 50$)
 - Memory overhead negligible
 - Simpler code may have less implementation overhead
- Algorithm requirements
 - Floyd-Warshall all-pairs shortest paths
 - Some algorithms naturally operate on matrices

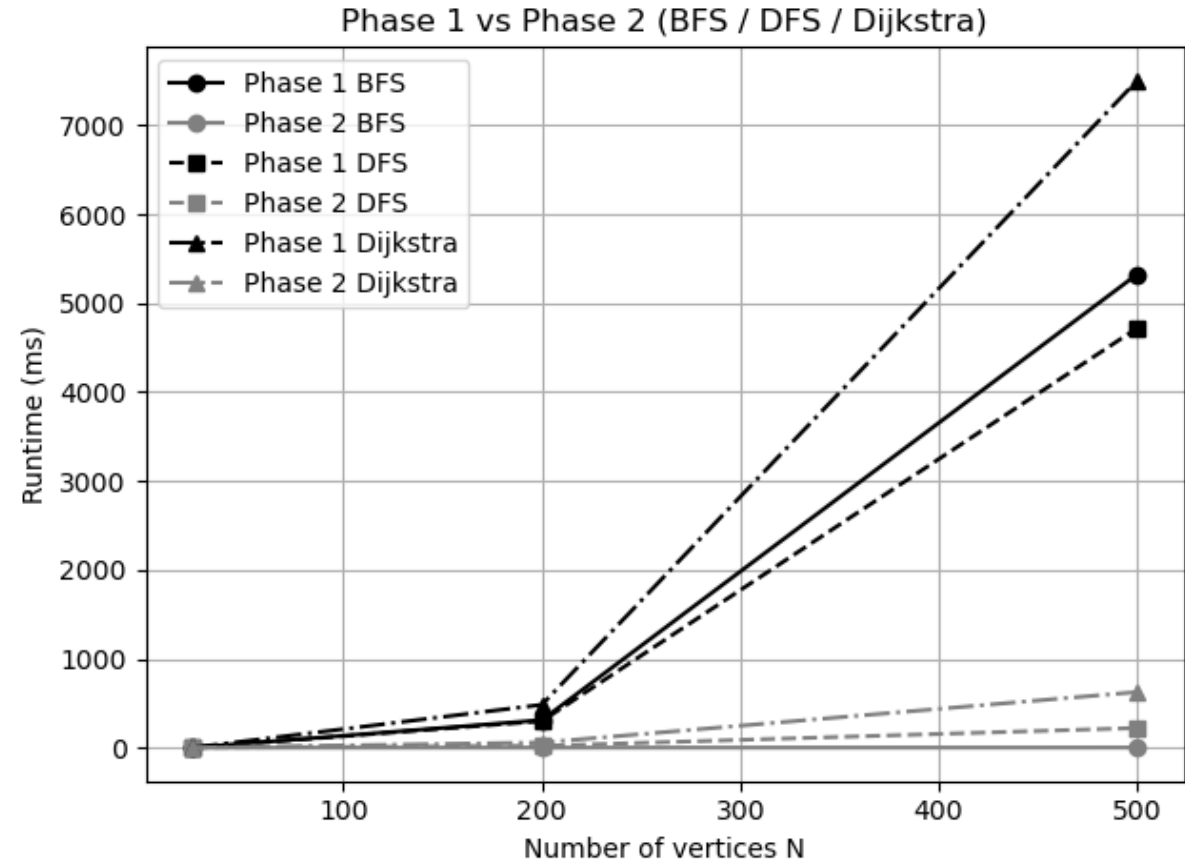
Adjacency Matrix

Dataset	N	BFS	DFS	Dijkstra
Small	24	0.861	0.744	1.164
Medium	200	310.452	295.509	478.394
Large	500	4770.935	4697.438	7393.170



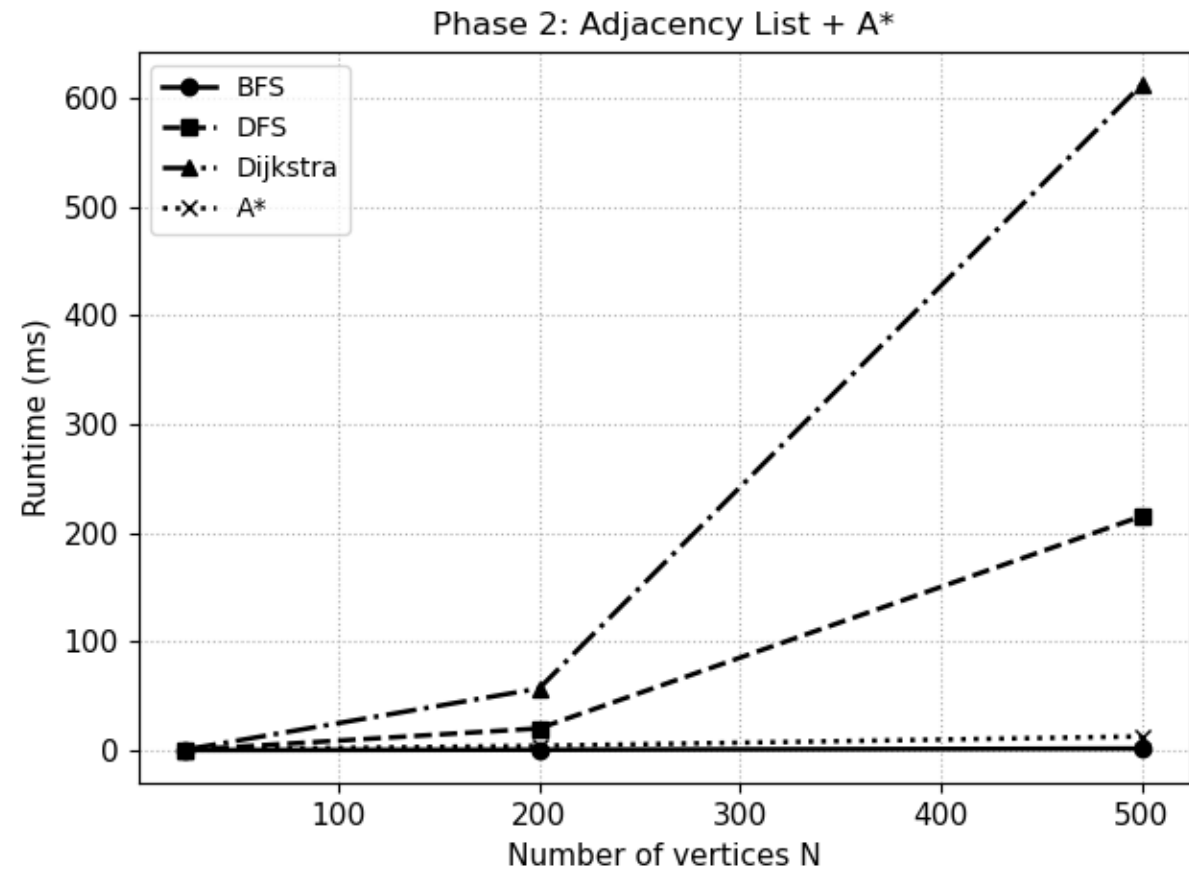
Adjacency List

Dataset	N	M	BFS	DFS	Dijkstra	A^*
Small	24	216	0.012	0.202	0.531	0.488
Medium	200	2966	0.280	19.965	56.721	3.965
Large	500	14987	1.436	215.130	611.833	12.593



Euclidean Heuristic

- Admissible
- Consistent



Euclidean Heuristic Algorithm Design

```
def astar(src, des, adj, coords_idx):
    n = len(adj)
    g = [INF] * n
    prev = [-1] * n
    g[src] = 0.0
    pq = []
    h0 = heuristic(src, des, coords_idx)
    heapq.heappush(pq, (h0, 0.0, src))

    while pq:
        f, cur_g, u = heapq.heappop(pq)
        if u == des:
            break
        if cur_g > g[u]:
            continue

        for v, w in adj[u]:
            new_g = g[u] + w
            if new_g < g[v]:
                g[v] = new_g
                prev[v] = u
                hv = heuristic(v, des, coords_idx)
                f_v = new_g + hv
                #use the sum of Euclidean length from curre
                heapq.heappush(pq, (f_v, new_g, v))

    return g, prev
```

$$O((V + E) \log V)$$

K shortest paths

```
A = [p0] # founded shortest paths  
B = []   # candidate paths (min-heap on total weight)
```

$$O(k \cdot V \cdot (V + E) \log V)$$